

# **Cyber Security Project**

## **Website Security Verification Tool**

**Name : Navneet kaur**

**Course : BSc. Computer sci.**

**Title**

Phishing Website Detection System

**Project Name**

Website Security Verification Tool

**Objective**

The objective of this project is to detect potentially phishing websites by analyzing URL characteristics and identifying suspicious patterns. The system helps users avoid fraudulent websites that attempt to steal sensitive information such as usernames, passwords, and banking details.

**Problem Statement**

Phishing attacks are among the most common cyber threats. Attackers create fake websites that closely resemble legitimate websites to trick users into revealing confidential information. Many users find it difficult to distinguish between genuine and phishing websites. Therefore, an automated system is needed to identify suspicious websites and warn users before they access them.

**Research Findings**

- Phishing websites often use unusual URLs, excessive special characters, or shortened links.
- Many phishing URLs contain IP addresses instead of domain names.
- Attackers frequently use misspelled versions of popular website names.
- Machine learning and URL-based analysis are commonly used techniques for phishing detection.
- Early detection of phishing websites can significantly reduce cybercrime and data theft.

**Methodology**

1. Study common characteristics of phishing websites.
2. Collect examples of legitimate and phishing URLs.
3. Identify URL-based indicators of phishing attacks.
4. Develop a detection system that analyzes URLs.
5. Classify URLs as Safe or Suspicious.
6. Test the system with multiple website links.

**Tools Used**

- Python 3
- Regular Expressions (re)
- URL Parsing Libraries
- VS Code / PyCharm
- Dataset of Legitimate and Phishing URLs (for testing)

## Implementation

### Python Code

```
import re

def detect_phishing(url):
    phishing_score = 0

    # Check for IP address in URL
    if re.search(r'https?://\d+\.\d+\.\d+\.\d+', url):
        phishing_score += 1

    # Check for @ symbol
    if '@' in url:
        phishing_score += 1

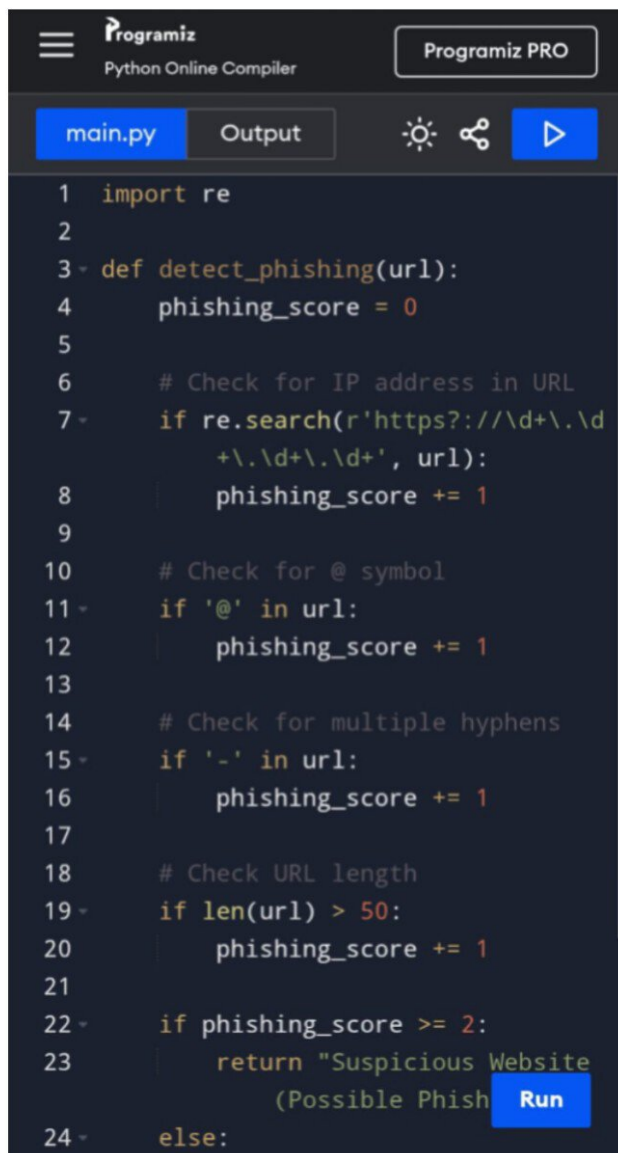
    # Check for multiple hyphens
    if '-' in url:
        phishing_score += 1

    # Check URL length
    if len(url) > 50:
        phishing_score += 1

    if phishing_score >= 2:
        return "Suspicious Website (Possible Phishing)"
    else:
        return "Safe Website"

url = input("Enter Website URL: ")
result = detect_phishing(url)

print("\nAnalysis Result:")
print(result)
```

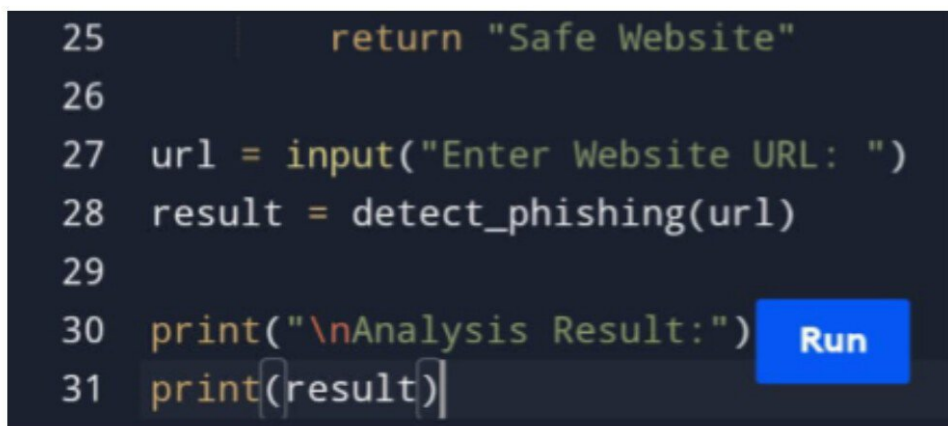


The screenshot shows the Programiz Python Online Compiler interface. At the top, there is a logo for Programiz, the text "Python Online Compiler", and a "Programiz PRO" badge. Below the header, there are tabs for "main.py" and "Output", along with icons for settings, share, and a play button. The main area contains Python code for a phishing detection function. The code defines a function `detect_phishing(url)` that calculates a phishing score based on several criteria: the presence of an IP address in the URL, the presence of an '@' symbol, the presence of multiple hyphens, and the URL length. If the score is 2 or higher, it returns "Suspicious Website (Possible Phish)", otherwise it returns "Safe Website". A blue "Run" button is visible at the bottom right of the code editor.

```
1 import re
2
3 def detect_phishing(url):
4     phishing_score = 0
5
6     # Check for IP address in URL
7     if re.search(r'https?://\d+\.\d+
8         \.\d+\.\d+', url):
9         phishing_score += 1
10
11    # Check for @ symbol
12    if '@' in url:
13        phishing_score += 1
14
15    # Check for multiple hyphens
16    if '-' in url:
17        phishing_score += 1
18
19    # Check URL length
20    if len(url) > 50:
21        phishing_score += 1
22
23    if phishing_score >= 2:
24        return "Suspicious Website
25            (Possible Phish"
26    else:
```

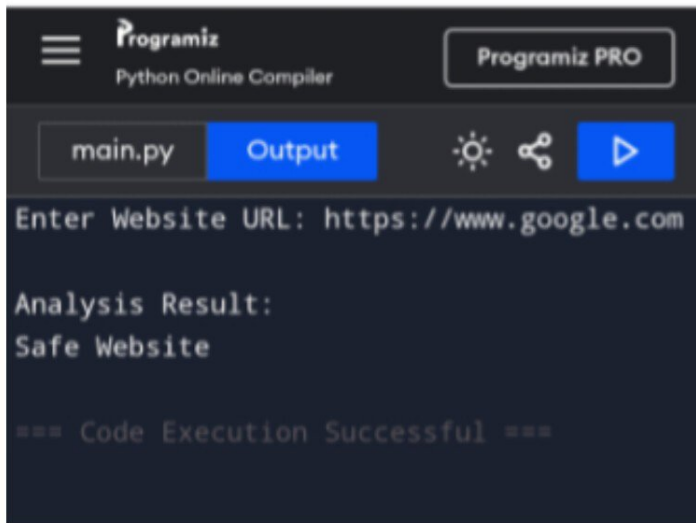
# Screenshots

## Python code

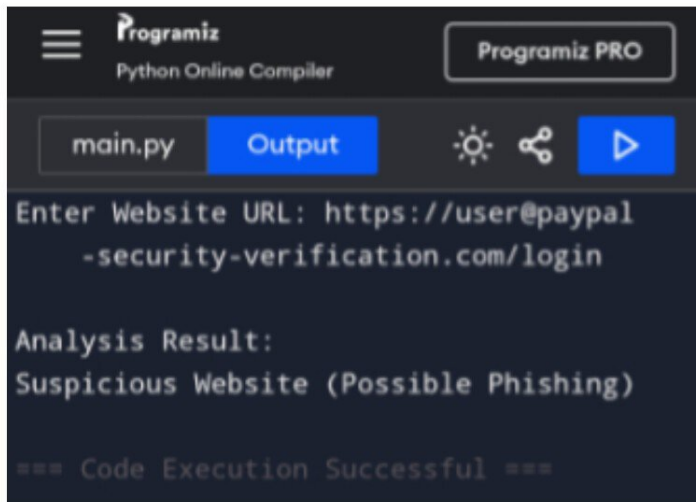


This screenshot shows the continuation of the Python code from the previous block. It includes the `return "Safe Website"` statement for the `detect_phishing` function, followed by the main execution logic. The code prompts the user to enter a website URL, calls the `detect_phishing` function, and prints the analysis result. A blue "Run" button is visible at the bottom right of the code editor.

```
25         return "Safe Website"
26
27 url = input("Enter Website URL: ")
28 result = detect_phishing(url)
29
30 print("\nAnalysis Result:")
31 print(result)
```



# Screenshot Output



## Results

- Successfully analyzed website URLs for phishing indicators.
- Identified suspicious URLs based on predefined security rules.
- Provided instant feedback to users regarding website safety.
- Demonstrated practical application of cybersecurity concepts in phishing prevention.

## Challenges

- Some legitimate websites may contain characteristics similar to phishing URLs.
- Advanced phishing websites can bypass simple detection rules.

- Maintaining high detection accuracy requires continuous updates and testing.

### **Future Scope**

- Integrate Machine Learning algorithms for improved detection accuracy.
- Develop a browser extension for real-time phishing protection.
- Add domain reputation and SSL certificate verification.
- Connect with online threat intelligence databases.
- Create a graphical user interface for better usability.

### **Conclusion**

The Phishing Website Detection System is an effective cybersecurity project that helps users identify potentially fraudulent websites. By analyzing URL patterns and suspicious indicators, the system provides an additional layer of protection against phishing attacks. The project demonstrates how cybersecurity techniques can be applied to enhance online safety and protect sensitive information from cybercriminals.